

作业10指导 时间片轮转调度，sleep系统调用 和 盘交换区的使用

同济大学计算机系操作系统 国豪2023 拔尖班2023
姓名 学号

Part 1、Unix V6 时间片轮转调度的实现

习题： Unix V6++系统中存在 3 个 CPU bound 用户态进程 PA、PB 和 PC。3 个进程静态优先数相等：100，p_cpu 是 0。Process[8]、[5]、[9]分别是 PA 、PB、PC 进程的 PCB。T 时刻是整数秒，PA 先运行。 注意 Process 数组的下标。

参考答案：

1、填表 select ()函数挑选就绪进程时，从lastSelect下标开始找

p_cpu	T	T+1	T+2	T+3	T+4	T+5
PA	0	40	20	0	40	20
PB	0	0	0	40	20	0
PC	0	0	40	20	0	40

p_pri	T	T+1	T+2	T+3	T+4	T+5
PA	100	102	101	100	102	101
PB	100	100	100	102	101	100
PC	100	100	102	101	100	102

2、画进程运行时序图。



3、T+1时刻，PA进程用完时间片放弃CPU。何时，PA进程会再次得到运行机会？简述T+1时刻系统怎样保护PA进程的用户态CPU执行现场，下次再运行时系统如何恢复PA进程的用户态CPU执行现场。

答：T+3时刻，PA进程会再次得到运行机会。T+1时刻系统保护PA进程的用户态CPU执行现场，T+3时刻恢复PA进程的用户态CPU执行现场的具体过程图示如下：

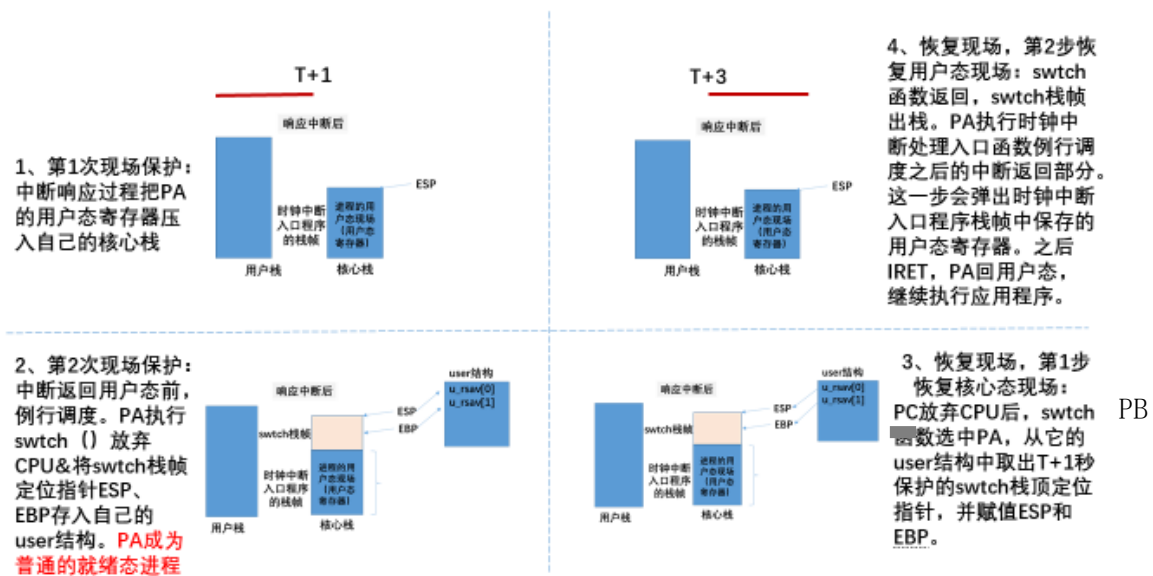
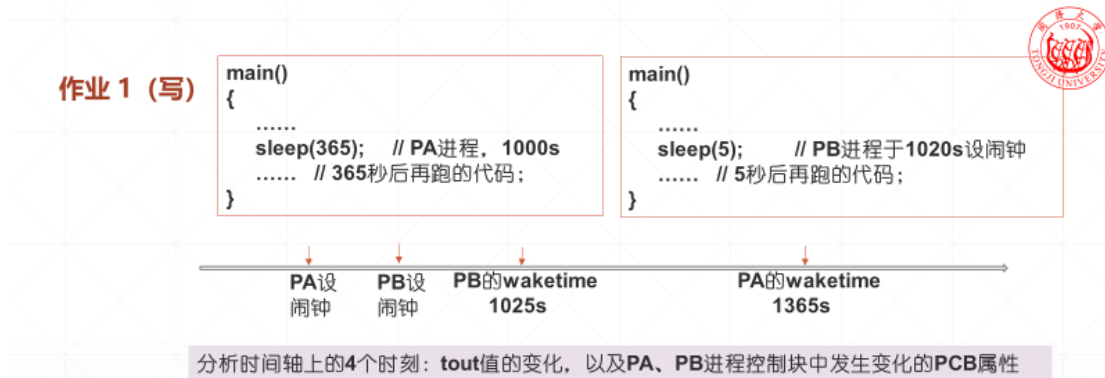


图 1

期末总复习：这是Unix系统，应用程序轮流使用CPU的方式。

Part 2、定时器服务



例题：已知，1000s之前，没有定时器未到期进程。结合上图中的4个时刻，看懂sleep(seconds)系统调用。

```

int SystemCall::Sys_Ssleep() // 系统调用sleep( seconds)
{
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    unsigned int wakeTime = Time::time + u.u_arg[0]; /* u.u_arg[0]是seconds变量的值 */
    while( wakeTime > Time::time )
    {
        if ( Time::tout <= Time::time || Time::tout > wakeTime )
        {
            Time::tout = wakeTime;
        }
        u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEEP);
    }
    X86Assembly::STI();

    return 0; /* GCC likes it ! */
}

void Time::Clock( struct pt_regs* regs, struct pt_context* context )
{
    .....
    if ( Time::time == Time::tout )
        procMgr.WakeUpAll((unsigned long)&Time::tout);
    .....
}

```

图 1

时刻 1000s: PA 执行 sleep 系统调用。(1) 设定时器, wakeTime=1365s (2) 修正定时器管理子系统状态: tout<=time (未发现未到期的定时睡眠进程), 赋值 tout = waketime 1365s (3) 入睡: p_stat = SWAIT, p_wchan = &tout, p_pri = 90。放弃 CPU, 成为低优先级睡眠进程。

时刻 1020s: PB 执行 sleep 系统调用。(1) 设定时器, wakeTime=1025s (2) 修正定时器管理子系统状态: tout>wakeTime (存在未到期的定时睡眠进程, 自己的 wakeTime 更早些), 重置 tout= waketime 1025s (3) 入睡: p_stat = SWAIT, p_wchan = &tout, p_pri = 90。放弃 CPU, 成为低优先级睡眠进程。

时刻 1025s: (1) 时钟中断处理程序唤醒 PA、PB 进程。PA、PB 的 PCB: p_stat = SRUN, p_wchan = 0, p_pri 不变还是 90。RunRun 加 2 次, 等于 2。现运行进程中断返回时, 被剥夺, 让出 CPU。Select() 函数选中 PA、PB, 其中离搜索起点较近的那个进程。不失一般性, PA。(2) PA 执行 sleep 系统调用的后半部分: wakeTime(1365)>time(1025), 未到苏醒时刻, 修正定时器管理子系统状态后再睡 (使用时刻 1000s, 红色步骤)。放弃 CPU 后, Select() 函数选中 PB 进程, Curpri==90 (3) PB 执行 sleep 系统调用的后半部分: wakeTime==time, 定时器到期, sleep 系统调用完成, 欲返回用户态。所以, 执行 Setpri() 函数计算、重置自己用户态的优先数 p_pri (不小于 100), 这个值大于 Curpri, RunRun++, 激活 select() 函数选择最高优先权的就绪进程。

时刻 1365s: (1) 时钟中断处理程序唤醒 PA 进程。PA 的 PCB: p_stat = SRUN, p_wchan = 0, p_pri 不变还是 90。RunRun 加 1 次, 等于 1。现运行进程中断返回时, 被剥夺, 让出 CPU。Select() 函数选中 PA (2) PA 执行 sleep 系统调用的后半部分: wakeTime(1365)==time(1365), 苏醒时刻到, sleep 系统调用完成, 欲返回用户态。所以, 执行 Setpri() 函数计算、重置自己用户态的优先数 p_pri (不小于 100), 这个值大于 Curpri, RunRun++, 激活 select() 函数, 系统选一个优先权最高的就绪进程运行。

tout 不动。后面再有进程定时器到期的话, 就会发现 tout<time, 是个过期的; 它会将自己的 waketime 赋值给 tout, 就像 PA 进程 1000s 时刻一样。

(2) 请简明扼要描述1025s时刻, 与时钟中断和Sleep系统调用相关的中断响应和调度过程。
已完成

(3*) 定时器服务存在tout惊群现象。请 (1) 简述惊群现象的缺点 (2) 优化系统, 每次只唤醒定时器到期的进程。

tout到期时, 系统会唤醒所有设置了定时器的睡眠进程。未到期所有进程需要再次入睡。惊群会增加系统入睡、唤醒操作数量, 增加进程切换开销, 降低系统性能。

优化思路, 使用特定数据结构, 每次只唤醒到期的定时入睡进程。

分析 1025 时刻 (1) 系统调度操作 (2) Sleep 系统调用下半部对 tout 变量的维护。
PA执行sleep函数, 用了多久才返回?

期末总复习: 以此题为背景, 梳理系统调用执行细节。

- 1、怎样 保存恢复用户态现场; 入睡、被唤醒后如何保存、恢复进程系统调用执行现场
。。。请参考图1。。。。
- 2、并发的2个系统调用怎样比翼双飞

Part 3、综合题

例题: 某Unix V6++系统, 硬盘中断开中断运行, 响应高优先级中断请求。系统并发3个进程, PA, PB 和 PX (CPU bound)。900s, 进程PA执行sleep(100)入睡。998s, PB进程执行read系统调用, 读磁盘文件。1000s, 现运行进程PX正在执行应用程序。PA设置的闹钟到期、PB读取的磁盘文件数据IO结束。请问, 执行磁盘中断处理程序的进程是 (PX), 执行时钟中断处理程序的进程是 (PX)。试分析1000s, 系统详细的调度过程。先响应磁盘中断。
答:

PX执行硬盘中断处理程序, 其间嵌套执行时钟中断处理程序。

此次时钟中断, 先前核心态, 为了不干扰正在执行的硬盘中断处理程序, 计数操作完成后时钟中断返回。整数秒应该执行的所有事务, 延迟至下一次时钟中断。

硬盘中断处理程序唤醒PB, RunRun++, 硬盘中断返回前调用Swch(), 现运行进程PX将CPU让给PB。PB完成read()系统调用后半段, 欲返回用户态。所以, 执行Setpri()函数计算、重置自己用户态的优先数p_pri (不小于100), 这个值大于Curpri, RunRun++, 激活select()函数选择最高优先权的就绪进程。PX和PB都有可能被选中, PB可能性大点儿, 假设被选中的是它。

下一次时钟中断 (1/60秒以后), PB响应、补上次未完成事务、更新time、唤醒PA、RunRun++、中断返回调度、激活Select()函数选中PA执行sleep()系统调用后半段。系统调用返回时刻, 调度。随后, 无内核任务需要执行, PX、PA、PB轮流使用CPU, 近似时间片轮转。

我们的习题: 全嵌套中断处理模式: 低优先级中断处理程序运行时, 系统响应高优先级中断处理请求。已知, 时钟中断优先级高于磁盘中断优先级。假设: 900s, PA 进程执行 sleep (100) 入睡。998s, PB 进程执行 read 系统调用, 读磁盘文件。1000s, 现运行进程 PX 正在执行应用程序。PA 设置的闹钟到期、PB 读取的磁盘文件数据 IO 结束。分析 1000s, 系统详细的调度过程。分两种情况考虑:

- 1、先响应磁盘中断
- 2、先响应时钟中断

Part 4、盘交换区

- 一、定义
- 二、盘交换区的使用策略



二、系统实现



可执行存储器管理策略：

- (1) 内存可以容纳全部进程图像时，盘交换区不用。
- (2) 内存不足以容纳全部进程图像时，最优布局：SWAIT放盘交换区，腾空间给SRUN和SSLEEP。
- (3) 若内存不足以容纳全部SRUN和SSLEEP，这些进程轮流使用内存资源。

中级调度规则，p_time FIFO。优先换入盘交换区上驻留最久的SRUN进程；必要时，换出最先进入内存的进程。

三、情景分析，例题与习题

背景知识



3、SWAIT入睡，唤醒 0#进程，换出自己的进程图像

```
void Process::Sleep(unsigned long chan, int pri)
{
    if (pri > 0)
    {
        .....
        if (procMgr.RunIn != 0)
        {
            procMgr.RunIn = 0;
            procMgr.WakeupAll((unsigned long)&procMgr.RunIn);
        }
        .....
    }
}
```

PB唤醒0#进程后入睡放弃CPU。
0#进程上台，换出PB、换入PA（假设PB尺寸大于PA）。
0#进程入睡，PA上台运行。

```
Sched ( )
{
    goto loop;

    Sloop:
    RunIn++, Sleep(&RunIn);
    ↓
    loop: // 换入
    寻找盘交换区p_time最大的就绪进程 PA,
    其驻留时长
    if (不存在) // 等待换入任务
        RunOut++; Sleep(&RunOut); go
    else
        为PA分配内存;
        成功, goto found2
```



4、中断处理程序唤醒盘交换区上的进程 PD

```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    /* 唤醒系统中所有因chan而进入睡眠的进程 */
    for(int i = 0; i < ProcessManager::NPROC; i++)
        if( this->process[i].IsSleepOn(chan) )
            this->process[i].SetRun();
}

void Process::SetRun()
{
    this->p_wchan = 0;
    this->p_stat = Process::SRUN;

    if ( this->p_pri < procMgr.CurPri )
        procMgr.RunRun++;

    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeUpAll((unsigned long)&procMgr.RunOut);
    }
}
```

现运行进程执行中断处理程序唤醒 PD和0#进程，中断返回时被剥夺，放弃CPU。
0#进程上台（假设内存有足够空间容纳PD），换入PD后入睡放弃CPU。
PD上台运行执行系统调用下半段。。。

例题： 假设某UNIX 系统中只存在4 个进程，分别是：睡眠中的0#进程，正在CPU 上执行的pa 进程，内存中就绪状态的pb 进程（CPU bound，不会IO），盘交换区上低优先级睡眠的pc 进程。已知，当前时刻 $T=1000.5\text{ s}$ ，内存空间已满。请尽量详细地分析以下时刻系统中与进程调度和中断相关的行为，并修改下表中的相关字段。

- (1) 1000.5 s，现运行进程 pa 执行 read（读文件）系统调用
- (2) 1001.5 s，pc 等待的 IO 操作完成

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	103	执行	SLOAD	2
pb	10K	101	就绪	SLOAD	2
pc	60K	90	低睡	~SLOAD	3

解：

- (1) 1000.5 s，现运行进程 pa 执行 read（读文件）系统调用
1000.5s，现运行进程pa执行read（读文件）系统调用，高优先权入睡（p_stat = SSLEEP, p_pri = -50），放弃CPU。随后，系统选中pb进程，因为它是内存中就绪的唯一进程。表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	SLOAD	2
pb	10K	101	执行	SLOAD	2
pc	60K	90	低睡	~SLOAD	3

1001.0s，现运行进程 pb 执行整数秒时钟中断处理程序，会++所有进程的 p_time。修正用户态进程的优先数，pb此轮执行了 0.5s，p_cpu增加30，整数秒时减20。总的效果：p_cpu加了10。整除16后，p_pri可能是102，也可能是101。表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	SLOAD	3
pb	10K	101/102	执行	SLOAD	3
pc	60K	90	低睡	~SLOAD	4

(2) 1001.5 s, pc 等待的 IO 操作完成

1001.5 s。

- 现运行进程 pb 执行中断处理程序，唤醒 pc，同时唤醒 0#进程。
- 中断处理程序执行完毕后，pb 将 CPU 让给 0#进程执行 sched () 。
 - 为盘交换区上的 pc 进程分配内存空间，未果。
 - 找内存中低睡 (SWAIT) 或 SSTOP (暂停) 的进程，未果。
 - pc, p_time==4s, 执行高代价对换操作。淘汰内存中的 pa 进程。具体而言，0#进程换出 pa、换入 pc (放在 pa 原先占据的内存区域) *。
- 完成对换操作后，0#进程 sleep(&RunOut,-100)入睡，将 CPU 让给内存中就绪的 pc 进程。pc 执行系统调用后半部。p-time 清 0。

表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	~SLOAD	0
pb	10K	101/102	就绪 SRUN	SLOAD	3
pc	60K	90	执行 SRUN	SLOAD	0

- 系统调用执行完毕后，pc 和 pb 时间片轮转，轮流执行应用程序。

表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	~SLOAD	0
pb	10K	101/102	SRUN	SLOAD	3
pc	60K	100+	SRUN	SLOAD	0

pb、pc 都有可能是现运行进程。

*处的细节，不要求掌握。就是换入、换出过程中，0#进程会睡眠等待IO结束的，此时pb使用CPU，是现运行进程。一旦IO完成，pb立即将CPU让给0#进程，因为它优先级最高。

习题： T0 时刻，某 UNIX V6++系统进程状态如下表所示。内存空间已满，除图示空间外，其余空间不可用。请尽量详细地分析以下时刻系统中与进程调度和对换操作（swap in, swap out）相关的行为，并修改下表中的相关字段。

序号	占用空间	状态	位置	内存起始物理地址
0#	-	高 睡 (RunOut)	SLOAD/SSYS	0x003ff000(4M 核心空间最后一个物理页框)
p1	40K	低睡	SLOAD	0x00408000
p2	30K	执行	SLOAD	0x00430000
p3	30K	低睡	~SLOAD	0x00450000

注释：高亮部分，盘交换区上的进程，内存起始地址字段答题不需要。是上一次在内存，为进程分配的内存空间的首地址，留着它有用的。

(1) T0 时刻（非整数秒），现运行进程 p2 执行 read 系统调用读磁盘文件（磁盘高速缓存中没有 p2 需要的文件数据）

(2) T1（T1=T0+1 秒）时刻，已完成 read 系统调用的 p2 进程运行在用户态。p3 等待的 I/O 操作完成。